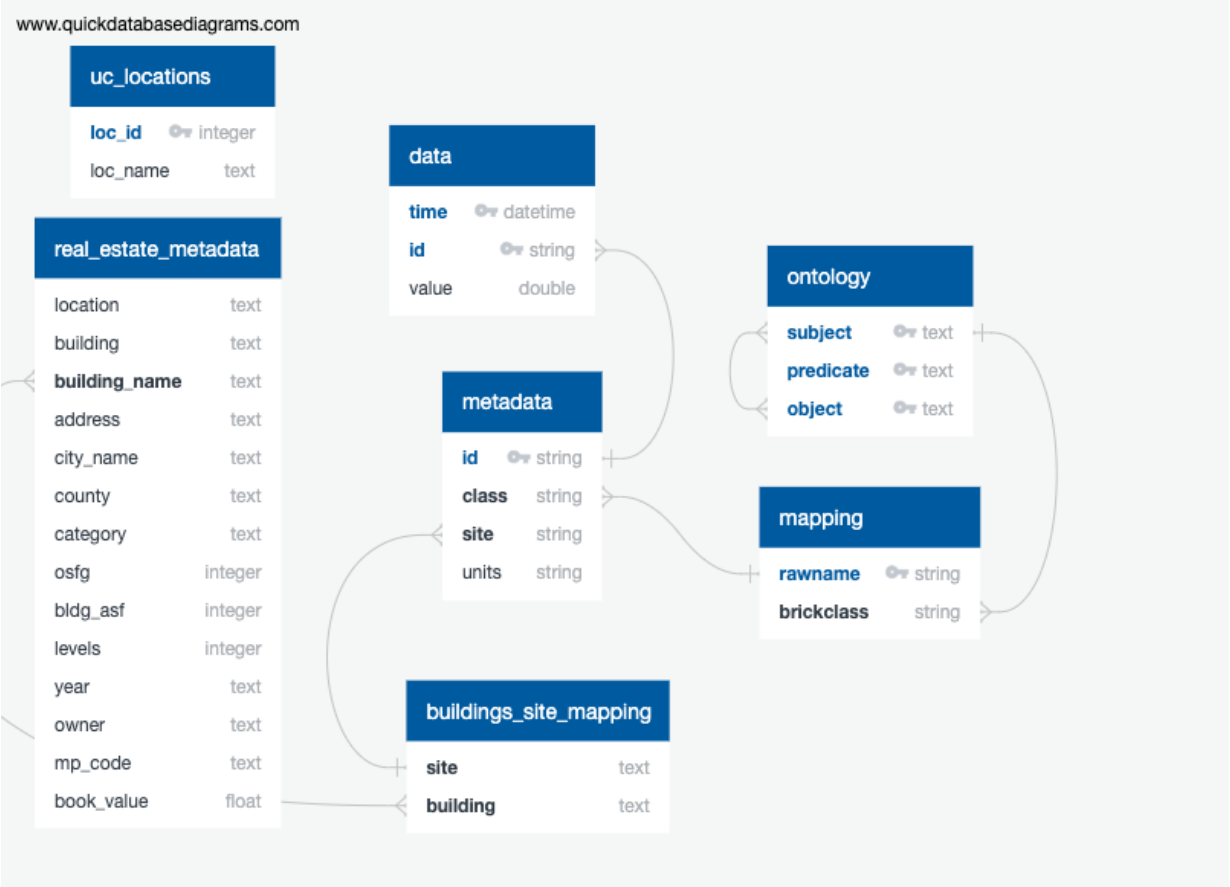
Data-Eng Project 5

Ruiji Sun (PhD Student in Building Science and Technology)
Yash Baldawa (Undergraduate in Computer Science, and Data Science)

Overview

What dataset did you use and where did you get it? Why was this dataset of particular interest to you?

- We use the Berkeley buildings dataset provided in class
- The dataset uses Brick ontology, which is interesting. The problem of most Building Automation Systems(BAS) is that the naming of the data across different vendors is not uniform, so it is literally difficult to get desired data from buildings. Therefore, we want to use the dataset with Brick ontology and do some analysis of building data.



What was the initial goal of your data pipeline? i.e., what problems did you plan to solve? What hypothesis were you trying to prove?

- The initial goal of our data pipeline is to extract the temperature data of a specific space in a building.The problem we plan to solve is that some spaces are very cold in the summer and very hot in the winter, which make occupants much uncomfortable and waste energy. We want to figure out the coldest space in the building as well as the warmest space in the building based on the temperature and date data.
- The hypothesis we were trying to prove is that the efficiency of HVAC systems and level of comfort based on the indoor temperature can be projected and attended to. HVAC systems can also be configured accordingly to adjust to varying numbers of foot traffic in the building. The problems we are trying to solve for this hypothesis via queries :
    a. Are people in this building comfortable?
    b. Is this building wasting energy? (HVAC systems operates when there are no occupants in spaces)
    c. Which building has the highest average indoor temperature?
    d. Which building has the lowest average indoor temperature?
- However, with the limited data available in the ucb_buildings database (actually there is no temperature data), we ended up calculating the energy usage index(EUI) of that building which is used extensively to certify the energy efficiency of a building. This in turn can be compared to the projected EUI of the building and make an initial hypothesis if the building is operating efficiently or not and sensors that might need to be installed to inspect the HVAC systems in the building.
- The data engineering problem we have with the data is that it is hard to determine if an HVAC system is operational in a part of a building or not based on the number of individuals present in the building. There will have to be multiple joins and aggregations and ensure deduplications and a ‘single source of truth’. For example, the occupancy sensor data cannot be understood by HVAC systems. Cooling or heating a space that doesn’t have any person would waste energy.

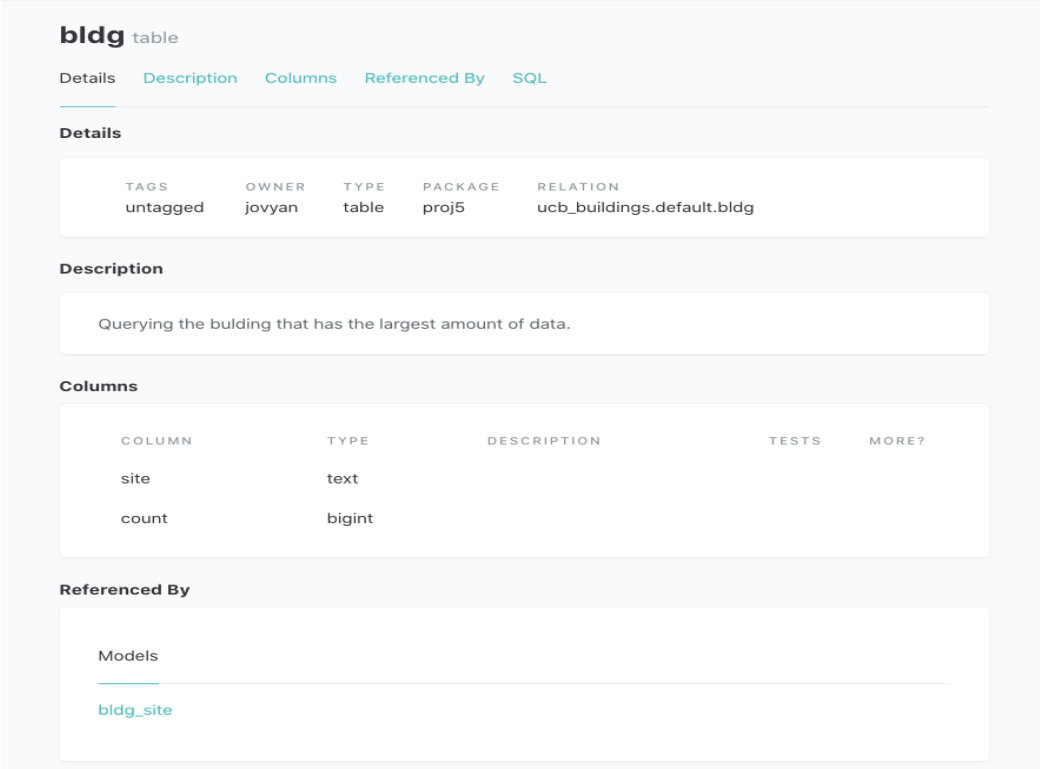
Model Structure

How much time did it take you to write your data pipeline? How long did the pipeline take to run?

- It does not take a substantial amount of time for us to write our data pipeline, since we used the fairly small ucb_buildings dataset available on datahub.
- The amount of time for running the pipeline is 6.69 s ± 132 ms per loop (mean ± std. dev. of 7 runs, 1 loop each), using timeit magic functions in jupyter notebook.

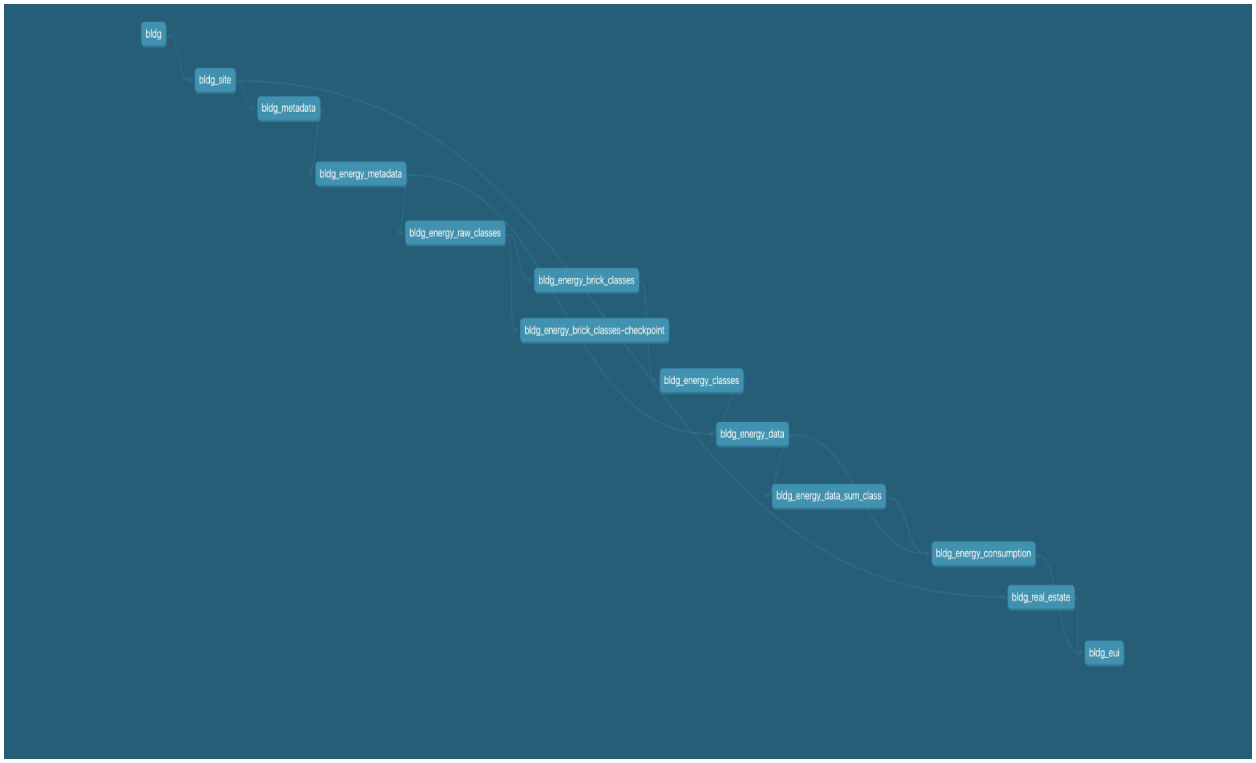
dbt specifications

Per the required specification, we added the descriptions in the project.yml folder. The image shown below is a description for the bldg table created from our first query. Description for all other tables are stored in the schema.yml file in the models folder



Lineage DAG diagram

Link to original image: https://photos.app.goo.gl/HXj9QEYnh8u3apEQ6



Data Quality tests

Tests can be found in https://github.com/ruijis/info290_dbt/tree/master/tests

check_building_count.sql	Checks after first query if checking if extra buildings crept in or buildings with same count, after use of aggregation functions
only_energy_units.sql	checking for non-energy consumption measurement units against those we know. List can be in exhaustive as new are discovered
eui_count.sql	checking for division against 0 / null values at the very end for the values extracted by previous window functions

Reflection

Did the outcome of your pipeline match your initial goals? If not, what caused you to change things?

- (Note: We expected to extract temperature data and humidity data. However, after careful initial inspection, only Wurster Hall had data available suitable for our calculations.)
- However, the pipeline to extract data just for Wurster hall, it did suit our needs and we were not able to obtain the required value.

If you had more time, what would you build on for this project?

- Some issues persisted with deploying infrastructure and understanding yaml files and jinja notation. We would like to go deeper into these aspects since deploying microservices in the cloud will also require an understanding of YAML templates/format.
- We used a postgres database already available on datahub. Using the original dataset using Big Query would have been much more interesting, however, we were looped into problems with setting up dbt in the first place that we couldn't take it to that step.
- Using Big Query would also require us to write more robust queries for data present.
- There is a dataset called Mortar, which has more than 100 buildings data, and we would like to work with that one.

How was your experience working with dbt? Thoughts on using it in future?

- dbt came off as a tool which might be hard to understand and deploy at first given how unique and different it is in setting up data flows.
- It should also be noted that public forums such as stackoverflow ***do not*** have enough support for dbt based issues such as notation, formatting, etc.
- dbt support is provided via it's slack community which may or may not be hard to navigate and access answers. However, once we had dbt set up and running, it was a breeze to get our project working.

If you had a graphical UI like Trifacta that would generate the SQL models and dbt specs, do you think you'd choose to work in the GUI or by writing your own SQL models and dbt specs?

For a project of the current scale, I will certainly prefer a UI based approach where my schema, models and tests did not need many customizations than my source database or data

warehouse. However, every time there has been a UI based tool used to generate and deploy infrastructure (Ex: AWS cloudformation GUI vs Cloudformation yaml templates to deal with circular dependencies), advanced tasks such as managing circular dependencies were more easily approachable via code. This is something I will need GUI based tools for.

Link to Code

Github Repository: https://github.com/ruijis/info290_dbt
Juptyter notebook displaying our SQL Queries - Nbviewer Jupyter Notebook
Link to view dbt workbook online -

```
#Note that we had to forcefully specify the location of profiles folder
#because we were not able to stage it in the root directory in datahub.
Github repo: https://github.com/ruijis/info290_dbt
Models test output on viewable jupyter notebook:
https://nbviewer.jupyter.org/github/yb4/info290_dbt/blob/master/pj5.ipynb
```

```
Commands to execute our project on Datahub after cloning github repo to
test on datahub:
rm -rf .ipynb_checkpoints/
dbt run --profiles-dir /home/jovyan/sp21/proj/proj5
dbt docs generate --profiles-dir /home/jovyan/sp21/proj/proj5
dbt docs serve --profiles-dir /home/jovyan/sp21/proj/proj5
```

Sources

https://docs.getdbt.com/docs/building-a-dbt-project/using-sources/

https://www.youtube.com/watch?v=Y03CsVDK69Y&ab_channel=KahanDataSolutions

DBT Slack Community -> advice-dbt-for-beginners/other channels

Connecting dbt to postgres

https://brickschema.org/

https://mortardata.org/